



Recibe una cálida:

¡Bienvenida!

Te estábamos esperando 😊 +

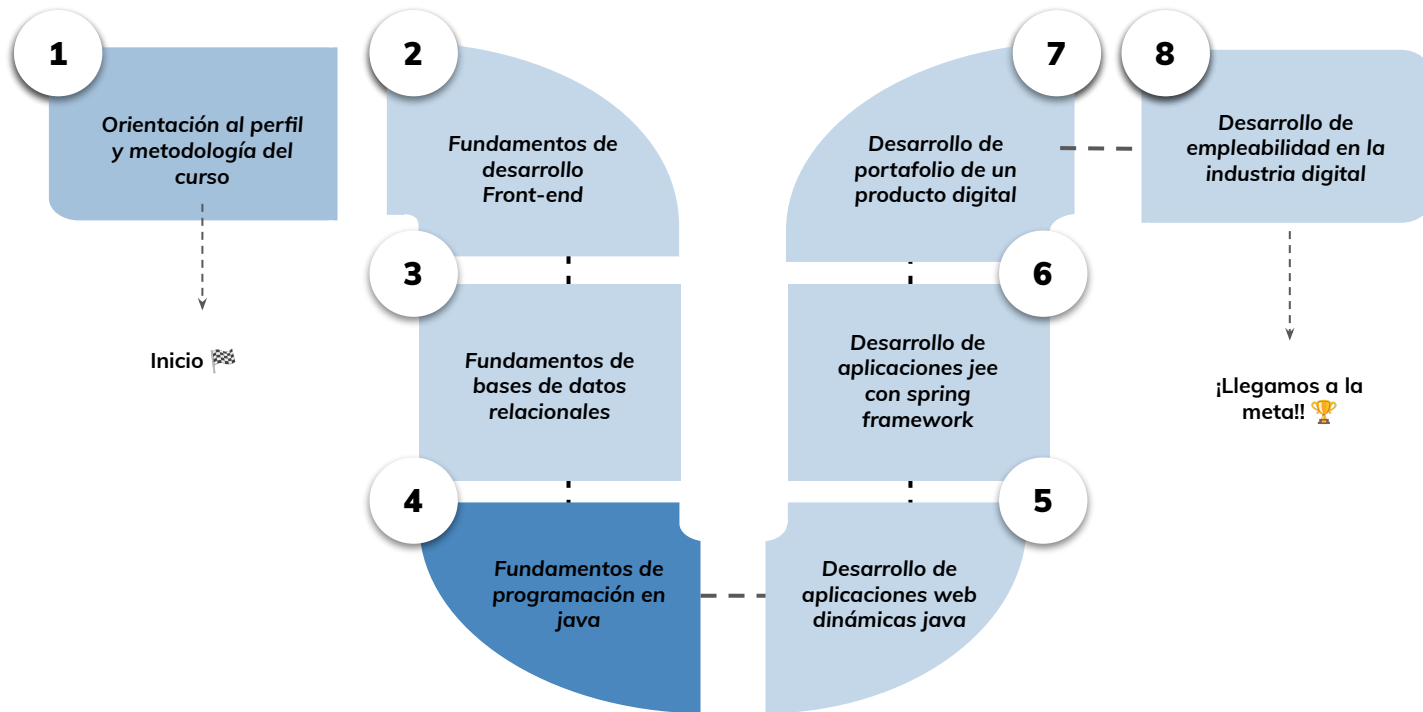


› Polimorfismo y principios básicos de diseño - Parte II

Aprendizaje Esperado: Utilizar principios básicos de diseño orientado a objetos para la implementación de una pieza de software acorde al lenguaje java para resolver un problema de baja complejidad.

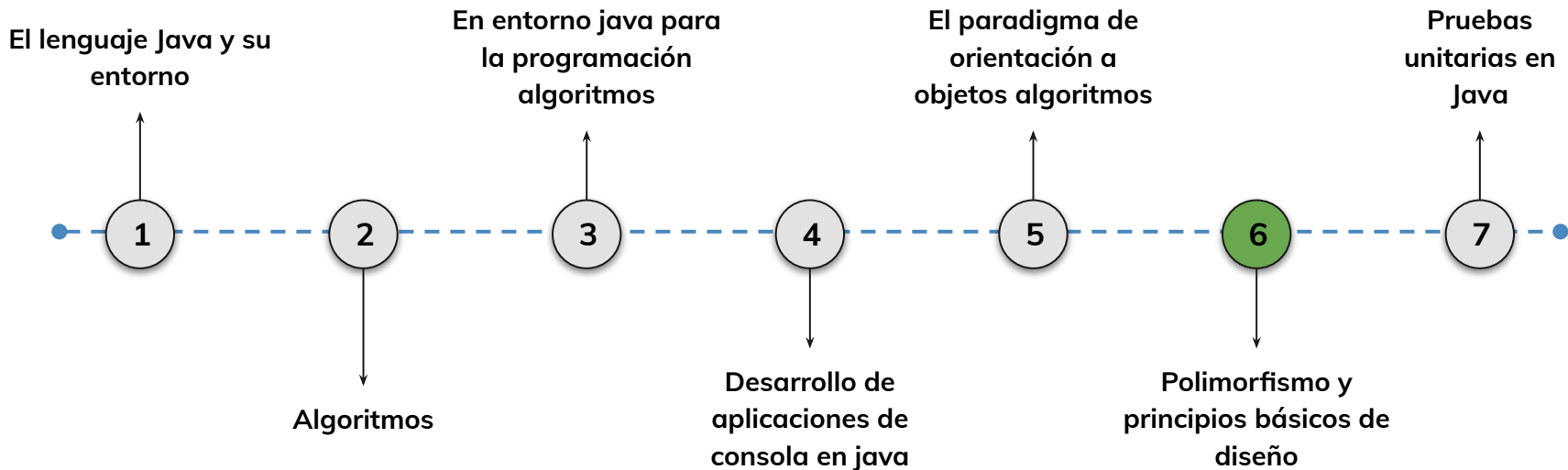
Hoja de ruta

¿Cuáles skills conforman el programa? **Desarrollo de Aplicaciones Full Stack Java Trainee**



Roadmap de lecciones

¿Cuáles **lecciones** estaremos estudiando en este módulo?



Learning Path

¿Cuáles temas trabajaremos hoy?

6.

Aplicación de principios SOLID y buenas prácticas de diseño en Java

En esta clase abordaremos conceptos clave del diseño orientado a objetos como cohesión, acoplamiento y reutilización. Además, estudiaremos los cinco principios SOLID, aplicándolos a través de ejemplos prácticos.

Principios básicos de diseño orientado a objetos

Cohesión, acoplamiento y mantenibilidad

Reutilización y refactorización de código

Principios SOLID

SRP, OCP y LSP (Responsabilidad única, Abierto/Cerrado, Sustitución de Liskov)

ISP y DIP (Segregación de interfaces e Inversión de dependencias)



Objetivos de aprendizaje

¿Qué aprenderás?



- Comprender los principios fundamentales de diseño orientado a objetos y su impacto en el desarrollo de software.
- Aplicar técnicas de cohesión y bajo acoplamiento en el diseño de clases.
- Reconocer y aplicar los cinco principios SOLID en el desarrollo con Java.
- Refactorizar código para mejorar la mantenibilidad y reutilización.
- Desarrollar software más robusto, flexible y escalable siguiendo buenas prácticas de diseño.

Repaso clase anterior

¿Quedó alguna duda?

En la clase anterior trabajamos :

- Comprendimos cómo se implementa la herencia en Java mediante `extends`.
- Aprendimos a utilizar `super()` para reutilizar constructores de la superclase.
- Analizamos el uso del polimorfismo con métodos sobrescritos e interfaces.
- Aplicamos clases y métodos abstractos para definir estructuras generales reutilizables.

➤ Principios de Diseño Orientado a Objetos

Principios de Diseño Orientado a Objetos

¿Qué son y para qué se utilizan?:

Un principio de diseño en Programación Orientada a Objetos (POO) es una regla o **guía general** que se utiliza para construir software de calidad y bien estructurado. Estos principios nos brindan directrices **para organizar y diseñar nuestras clases y objetos de manera efectiva**, promoviendo un **código limpio, mantenible y fácil de entender**.

Al seguir estos principios, podemos obtener múltiples **beneficios** en términos de **organización** del código, **mantenibilidad**, **reutilización** de código, **flexibilidad** y **calidad** del software en general.

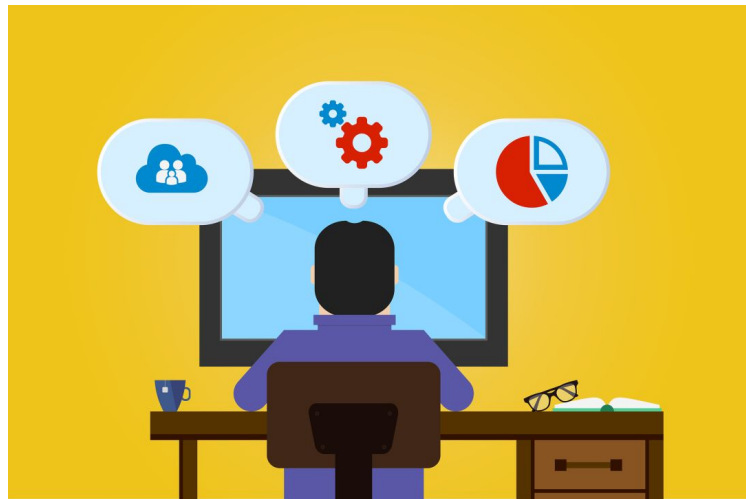
➤ Mantenibilidad

Mantenibilidad

d

Se refiere a la **capacidad** de un software **para ser modificado, corregido, mejorado o adaptado de manera eficiente y efectiva** a lo largo del tiempo.

- Código bien organizado, poco acoplado, con responsabilidades claras.
- Agregar features o arreglar bugs de forma rápida y sencilla.
- Depende de tener un diseño limpio, módulos independientes y una arquitectura escalable.
- Equipos pequeños pueden mantener grandes sistemas si están bien diseñados.
- Reducir duplicación y extraer abstracciones aumenta la mantenibilidad.



Mantenibilidad

La mantenibilidad es importante porque los sistemas de software evolucionan constantemente. Se requieren actualizaciones, correcciones de errores y mejoras en función de los cambios en los requisitos y las necesidades de los usuarios.

Esto facilita la comprensión del sistema, la identificación rápida de problemas y la implementación de cambios sin introducir nuevos errores.

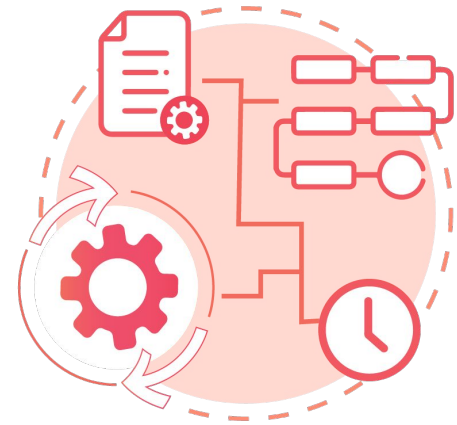


➤ Reutilizabilidad

Reutilizabilidad

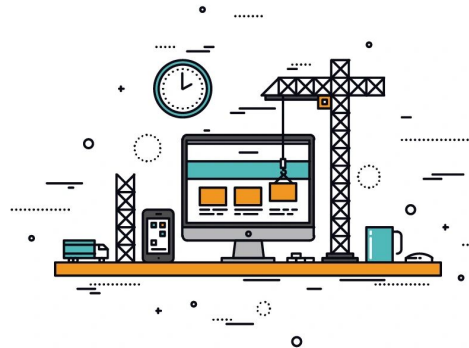
Se refiere a la capacidad de **utilizar componentes, módulos o partes de software existentes** en diferentes contextos o aplicaciones **sin tener que reescribirlos** desde cero.

- En lugar de reinventar la rueda, se crean piezas reutilizables.
- La POO permite crear clases y librerías paramétricas y genéricas.
- Las interfaces y la herencia facilitan reutilizar comportamientos.
- Se reduce redundancia y duplicación de esfuerzos.
- Menor cantidad de código implica menor costo de mantenimiento.
- Aumenta la productividad y simplicidad del software.



Reutilizabilidad

La reutilizabilidad se logra mediante el diseño e implementación de componentes y bibliotecas que son independientes, modulares y fácilmente integrables en diferentes sistemas. Esto implica utilizar técnicas como la encapsulación, la abstracción y la creación de interfaces claras y flexibles.



LIVE CODING

Reutilizando:

Vamos a tomar una clase específica y generalizarla para hacerla reusable y fácil de mantener. En este caso, cambiaremos de Cuenta a CuentaBancaria. Esto nos permitirá generalizar aún más el alcance de la Clase. También vamos a refactorizar código repetido de ser necesario.

Tiempo: 30 minutos

» Cohesión

Cohesión

La cohesión se refiere al **grado en que los miembros de una clase o componente de software están relacionados y trabajan juntos para lograr un objetivo específico**.

En otras palabras, una clase cohesiva se centra en una única responsabilidad o función.

Tipos de Cohesión:

- Cohesión **Alta**: Una clase con alta cohesión **realiza una sola tarea** o función bien definida. Es deseable, ya que **facilita el mantenimiento y la reutilización del código**.
- Cohesión **Media**: Una clase con cohesión media realiza varias tareas relacionadas, pero no de manera ideal. Puede ser necesario dividir la clase en clases más pequeñas y cohesivas.
- Cohesión **Baja**: Una clase con baja cohesión realiza tareas no relacionadas y suele ser difícil de entender y mantener.

Cohesión

Para lograr una alta cohesión:

- Asignar una única responsabilidad bien definida a cada clase.
- Mantener los elementos de una clase (atributos, métodos) enfocados en esa única responsabilidad.
- Dividir clases grandes en otras más pequeñas y cohesivas.

```
// Ejemplo de alta cohesión
class Calculator {
    public int add(int a, int b) {
        return a + b;
    }

    public int subtract(int a, int b) {
        return a - b;
    }

    public int multiply(int a, int b) {
        return a * b;
    }
}
```

Cohesión

- Agrupar elementos lógicamente relacionados en una misma clase.
- No incluir funcionalidades que no estén estrictamente relacionadas con la responsabilidad principal de la clase.
- Evitar clases "utilidad" para funcionalidad variada no relacionada.
- Revisar continuamente que las clases sigan siendo cohesivas frente a cambios.

```
// Ejemplo de alta cohesión
class Calculator {
    public int add(int a, int b) {
        return a + b;
    }

    public int subtract(int a, int b) {
        return a - b;
    }

    public int multiply(int a, int b) {
        return a * b;
    }
}
```

➤ Acoplamiento

Acoplamiento

El acoplamiento se refiere a **la medida en que dos o más clases están interconectadas o dependen entre sí**. **Un acoplamiento bajo es deseable**, ya que reduce la dependencia entre clases y hace que el código sea más flexible y fácil de mantener.

Tipos de Acoplamiento:

- Acoplamiento **Bajo**: Las clases tienen **poca o ninguna dependencia entre sí**. Cambios en una clase tienen un impacto mínimo en otras clases.
- Acoplamiento **Medio**: Las clases tienen alguna dependencia entre sí, pero no están completamente entrelazadas. Se puede mejorar para reducir la dependencia.
- Acoplamiento **Alto**: Las clases están fuertemente interconectadas y dependen en gran medida unas de otras. Cambios en una clase pueden afectar muchas otras clases.

Acoplamiento

Para lograr un bajo acoplamiento:

- Definir interfaces o abstracciones que permitan la interacción sin depender de las implementaciones concretas.
- Cada clase debe tener una única responsabilidad bien definida y no debe mezclar múltiples responsabilidades.
- Organizar el código en módulos o paquetes independientes con responsabilidades claras.
- Evitar dependencias rígidas, como llamadas directas a métodos de otras clases.
- Usar variables locales o limitar el alcance de las variables al mínimo necesario.
- Reducir el uso de variables globales que pueden aumentar el acoplamiento.
- Mantener la lógica de negocio separada de la interfaz de usuario para facilitar cambios y pruebas.

```
// Mal: Acoplamiento alto
class Order {
    private MySQLDatabase database;

    public Order() {
        database = new MySQLDatabase();
    }

    // ...
}

// Bien: Bajo acoplamiento
interface Database {
    void saveData();
}

class Order {
    private Database database;

    public Order(Database database) {
        this.database = database;
    }
}
```

Acoplamiento

LIVE CODING

Creando una Wallet cohesiva y de bajo acoplamiento:

Vamos a mejorar el diseño de la Wallet que gestiona el saldo y las transacciones de una billetera virtual. En este ejercicio, aplicaremos los principios de cohesión y acoplamiento para asegurarnos de que la clase sea modular y fácil de mantener.

Objetivo:

Diseñar la Wallet con alta cohesión y bajo acoplamiento para gestionar el saldo y las transacciones.

LIVE CODING

1. *La clase CuentaBancaria debe tener una única responsabilidad: gestionar el saldo y las transacciones.*
2. *La clase CuentaBancaria debe proporcionar métodos para:*
 - *Consultar el saldo actual.*
 - *Realizar depósitos.*
 - *Realizar retiros.*
 - *Registrar transacciones.*

LIVE CODING

3. *Crear una clase Transaccion con los atributos origen, destino y monto.*
4. *Crear un método realizar() para la clase Transaccion que permita realizar la transacción.*

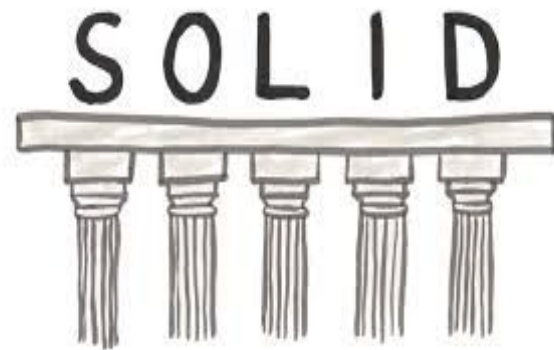
Tiempo: 40 minutos

➤ Principios SOLID

Principios SOLID

¿Qué son y para qué se utilizan?:

Los principios **SOLID** son 5 principios básicos de la programación orientada a objetos. Estos fueron desarrollados por Robert C. Martin (Uncle Bob) en el año 2000. Los principios SOLID **tienen como objetivo eliminar las malas prácticas en el diseño y desarrollo de software** . La aplicación de estos principios **ayudan al desarrollador a escribir un código mantenible, escalable y robusto** .



Principios SOLID

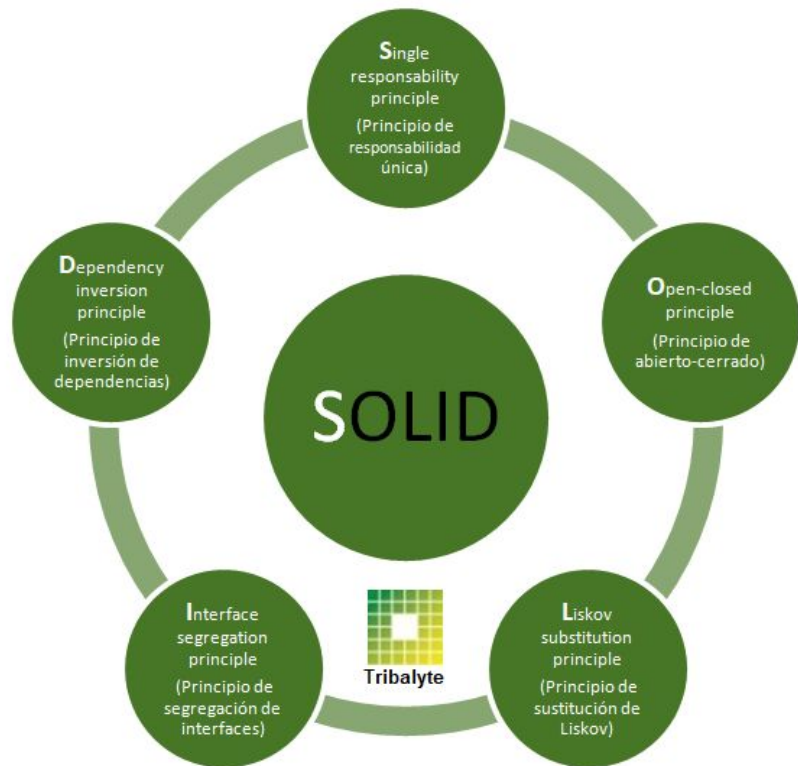
Los principios SOLID son:

- ↪ **Single Responsibility Principle** (Principio de responsabilidad única).
- ↪ **Open/Closed Principle** (Principio de abierto/cerrado).
- ↪ **Liskov Substitution Principle** (Principio de sustitución de Liskov).
- ↪ **Interface Segregation Principle** (Principio de segregación de interfaces).
- ↪ **Dependency Inversion Principle** (Principio de inversión de dependencia).

Principios SOLID

El término SOLID es un acrónimo de los 5 principios.

En esta clase veremos el primero de ellos.





Momento:

Time-out!



5 -10 min.





Rompehielo

Piezas interconectadas:

Como podemos ver, este objeto está formado por varias piezas que cumplen una función específica y suman al todo.

Respondan en el chat o levantando la mano:

- ¿Qué les sugiere esta imagen en términos de organización de componentes?
- ¿Cómo creen que se relaciona esta imagen con la programación Java?
- ¿Qué similitudes pueden encontrar entre la estructura de esta imagen y el código Java bien organizado?



» Principio de Responsabilidad Única

Responsabilidad Única

Este principio establece que **cada clase debe tener una única responsabilidad** dentro de nuestro software, y esta responsabilidad debe estar **definida y ser concreta** .

Todos los métodos deben estar alineados con la finalidad de la clase.

“Una clase debe tener solo una razón para cambiar” – Uncle Bob

Si nos encontramos con una **clase que dispone de métodos que realiza tareas con distintas finalidades se debería refactorizar el código** y crear clases nuevas que se corresponda con los objetivos del método.

Responsabilidad Única

Gracias a este principio nuestras clases tendrán un **bajo acoplamiento** y a la hora de realizar cambios menos clases se verán afectadas.

Algunos síntomas que nos pueden hacer sospechar (bad smells) que **no se está cumpliendo este principio** serían:

- Una clase es demasiado larga, demasiadas líneas de código.
- Cada vez que hay que introducir una modificación o una nueva funcionalidad, es necesario tocar en muchos sitios.
- Hay demasiadas relaciones entre clases.
- Mezcla funcionalidades de distintas capas de la arquitectura.



LIVE CODING

¡Poniendo en práctica!

Evaluar si la clase CuentaBancaria sigue el principio de responsabilidad única, centrándose únicamente en la gestión del saldo y las transacciones.

1. *Refactorizar según el principio de responsabilidad única*

Veamos un ejemplo parecido: 🙌

Tiempo: 30 minutos

LIVE CODING

```
public class CuentaBancaria{  
  
    private List<Coin> coins;  
  
    public void addCoin(Coin coin) {  
    }  
  
    public void sendPayment(double  
amount) {  
    }  
  
    public double getBalance() {  
    }  
}
```

```
    public void refreshPrices() {  
        // actualiza precios de mercado  
de las coins  
    }  
  
    public void showPortfolio() {  
        // muestra balance total y por  
coin  
    }  
  
    public double calculateTaxes() {  
        // calcula impuestos sobre  
ganancias  
    }  
}
```

» Principio de Abierto/Cerrado

Abierto/Cerrado

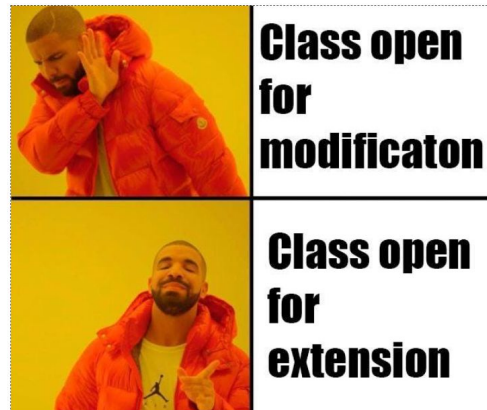
Este principio establece que una entidad de software (**clase, módulo, función** , etc) **debe quedar abierta para su extensión, pero cerrada para su modificación** .

Con **abierta para su extensión** , nos quiere decir que una entidad de software debe tener la **capacidad de adaptarse a los cambios** y nuevas necesidades de una aplicación, pero con la segunda parte de “**cerrada para su modificación** ” nos da a entender que la adaptabilidad de la entidad no debe darse como resultado de la modificación del core de dicha entidad si no **como resultado de un diseño que facilite la extensión sin modificaciones** .

Abierto/Cerrado

Las ventajas que nos ofrece diseñar el código aplicando este principio es un **software más fácil de mantener** :

- Minimiza los cambios en la base de código de la aplicación y amplía funcionalidades sin modificar partes básicas de la aplicación probadas.
- Evita generar nuevos errores en funcionalidades que ya estaban desarrolladas, probadas y funcionando correctamente.
- A la hora de implementar test unitarios nos ayudará a no tener que modificar dichos tests cada vez que se mejore y amplíe el código.
- Mejora la fiabilidad de nuestra base de código



LIVE CODING

Imaginemos que tenemos una wallet básica con la siguiente estructura:

```
public class Wallet {  
    public void payWithCreditCard(double amount){  
        //pago con tarjeta de crédito  
    }  
    public void payWithDebitCard(double amount){  
        //pago con tarjeta de débito  
    }  
    public void payWithPayPal(double amount){  
        //pago con PayPal  
    }  
}
```

LIVE CODING

Ahora, si el cliente nos informa que se necesita agregar un nuevo método de pago, como criptomonedas, deberíamos modificar la clase Wallet agregando un nuevo método. Esto es algo que deberíamos evitar (el modificar la clase).

Una buena alternativa para respetar el principio de Abierto/Cerrado sería crear una interfaz PaymentMethod:

```
public interface PaymentMethod {  
    void pay(double amount);  
}
```

De esta manera, se podrían crear nuevas clases de pago implementando la interfaz PaymentMethod sin alterar Wallet.

Tiempo: 30 minutos

➤ Principio de Liskov

Principio de Liskov

¿Qué es?

El principio de Liskov, también conocido como el Principio de Sustitución de Liskov (Liskov Substitution Principle o LSP por sus siglas en inglés), es uno de los cinco principios SOLID de la programación orientada a objetos (POO) y fue formulado por Barbara Liskov en 1987. El principio de Liskov establece una **regla fundamental para la herencia y la abstracción** en la POO y se resume de la siguiente manera:

"Los objetos de una clase derivada (subclase) deben ser capaces de reemplazar objetos de la clase base (superclase) sin afectar la corrección del programa".

Principio de Liskov

Algunos puntos clave:

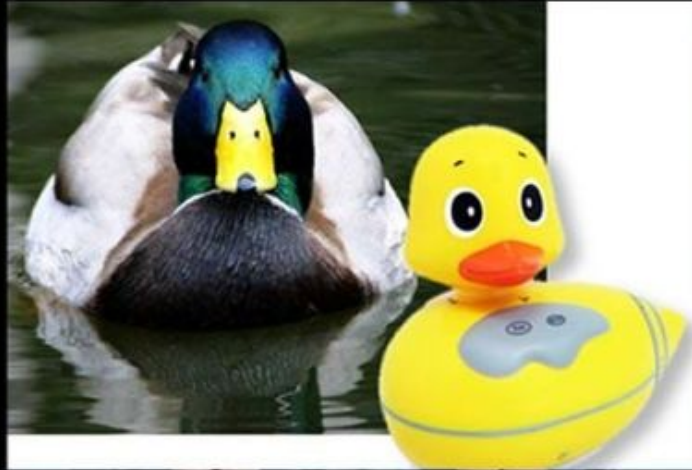
- **Subclases como Sustitutas** : Una subclase debe ser una sustituta válida de su superclase. Esto significa que debe proporcionar la misma interfaz (métodos) y cumplir las mismas expectativas de comportamiento que la superclase.
- **Herencia Correcta** : La herencia debe utilizarse de manera que refleje una relación "es un" entre la subclase y la superclase. Esto significa que una subclase debe ser una extensión lógica de la superclase y no debe alterar su semántica básica.
- **Invariantes Preservadas** : Una invariante es una propiedad que se mantiene constante en el tiempo. Si la superclase tiene invariantes, la subclase no debe romperlas ni modificarlas.



Principio de Liskov

- **Precondiciones y Postcondiciones** : Si la superclase tiene precondiciones (condiciones que deben cumplirse antes de que un método se ejecute) y postcondiciones (resultados esperados después de que un método se ejecute), entonces la subclase debe mantener esas condiciones.
- **Herencia Responsable** : La herencia debe usarse de manera responsable y reflexiva. No todas las relaciones entre clases deben modelarse como herencia, a veces es más apropiado usar composición u otras técnicas.

Principio de Liskov



Liskov Substitution Principle

If it looks like a duck and quacks like a duck but needs batteries, you probably have the wrong abstraction.

Principio de Liskov

El principio de Liskov nos da una serie de pautas para guiar la herencia entre clases. La principal que debe cumplir si estamos realizando la herencia de una manera correcta es que cada clase que hereda de otra puede usarse como su padre sin necesidad de conocer las diferencias entre ellas.

Si cuando sobreescribimos un método en la clase que hereda necesitamos lanzar una excepción o no realiza nada, entonces probablemente estamos violando el LSP .

Esta premisa es clave para detectar fallos en la arquitectura de nuestra aplicación o posibles “code smells”.

➤ Principio de Segregación de Interfaces

Segregación de Interfaces

¿Qué es?:

El principio de segregación de interfaces (ISP, por sus siglas en inglés: Interface Segregation Principle) es uno de los cinco principios SOLID de la programación orientada a objetos (POO). Este principio fue propuesto por Robert C. Martin y se centra en la estructuración adecuada de las interfaces en una aplicación. El ISP se formula de la siguiente manera:

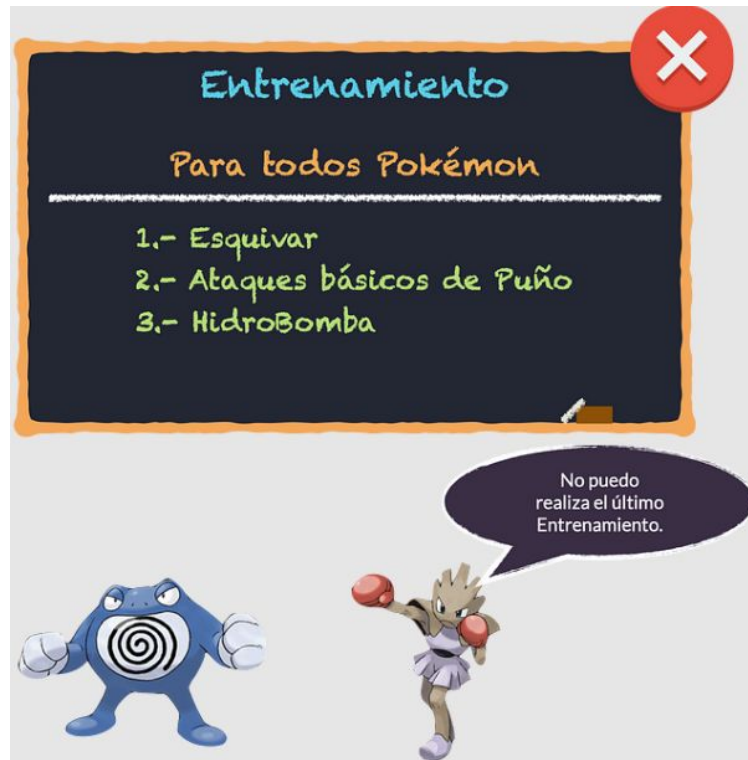
"Ninguna clase debe verse forzada a implementar métodos que no utiliza, y las interfaces deben ser específicas para las necesidades de las clases que las utilizan. "

El ISP se basa en la idea de que las **interfaces deben ser pequeñas, cohesivas y específicas** para cada clase que las implementa.

Segregación de Interfaces

Algunos conceptos clave:

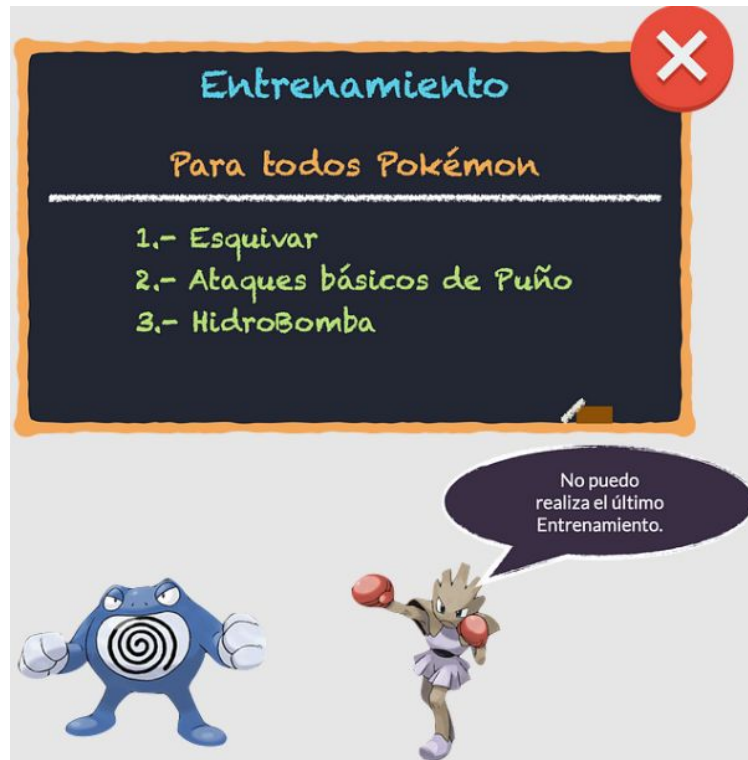
- **Interfaces Coherentes** : Los métodos en una interfaz deben estar relacionados en función del contexto y de las responsabilidades de la clase que la implementa.



Segregación de Interfaces

Algunos conceptos clave:

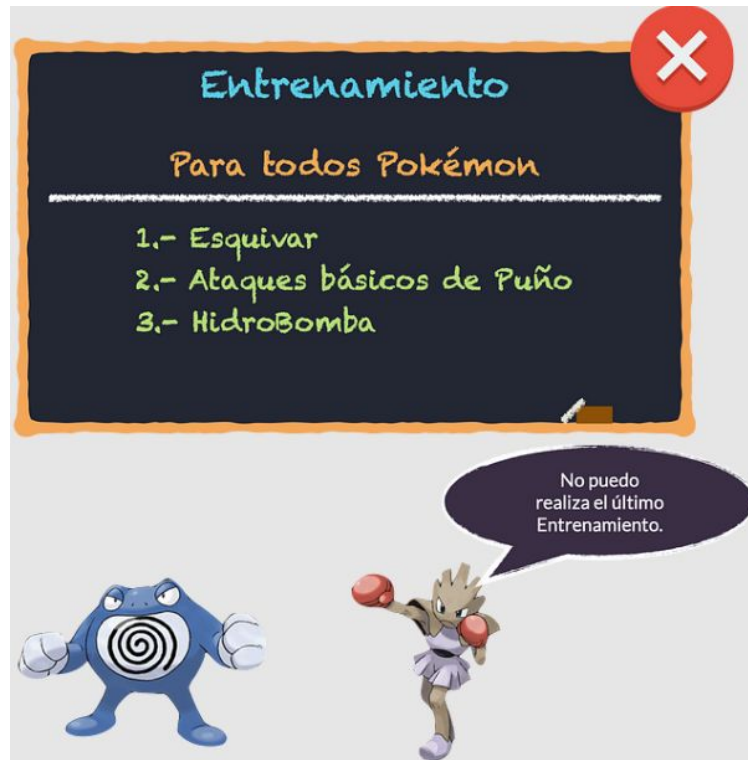
- **Evitar Interfaces Monolíticas** : Las interfaces grandes y complejas pueden obligar a las clases a implementar métodos que no necesitan, lo que va en contra del ISP.



Segregación de Interfaces

Algunos conceptos clave:

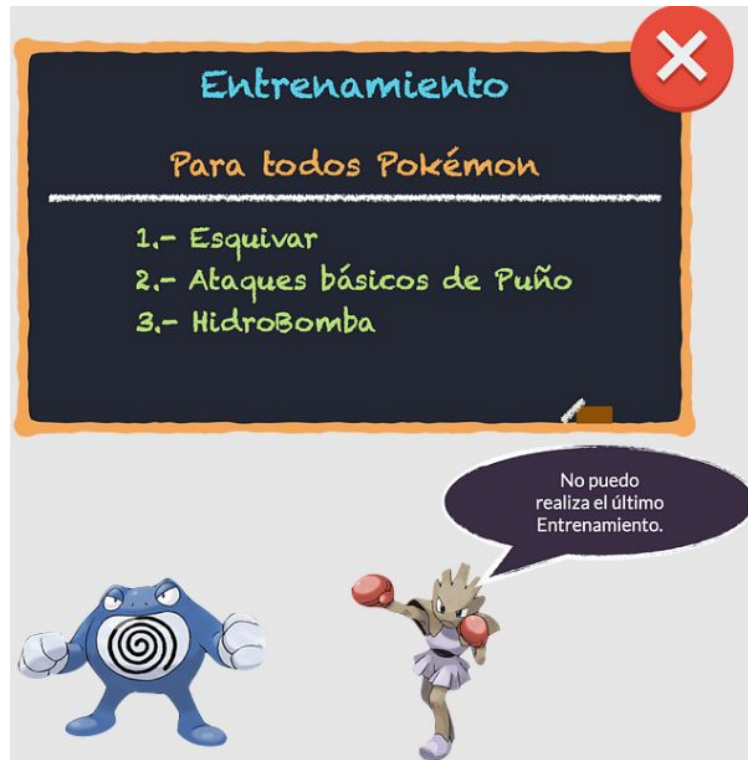
- **División de Interfaces** : Si una clase necesita diferentes funcionalidades o comportamientos, es preferible dividir las interfaces en interfaces más pequeñas y específicas.



Segregación de Interfaces

Algunos conceptos clave:

- **Aplicación en Herencia Múltiple** : Especialmente relevante en lenguajes de programación que admiten implementación de múltiples interfaces. En estos casos, las clases pueden seleccionar las interfaces que desean implementar de manera más granular.



Segregación de Interfaces

Beneficios de Cumplir con el ISP:

- **Mantenimiento Simplificado** : Las interfaces específicas y cohesivas facilitan el mantenimiento del código, ya que los cambios en una interfaz afectan solo a las clases que realmente la implementan.



Segregación de Interfaces

Beneficios de Cumplir con el ISP:

- **Reutilización Mejorada:** Las interfaces específicas hacen que sea más fácil reutilizar clases en diferentes contextos, ya que una clase puede implementar solo las interfaces relevantes para una situación particular.



Segregación de Interfaces

Beneficios de Cumplir con el ISP:

- **Flexibilidad** : Permite una mayor flexibilidad en la composición de objetos, ya que las clases pueden seleccionar las interfaces que necesitan.



➤ Principio de Inversión de Dependencias

Principio de Inversión de Dependencias

¿Qué es?:

El principio de inversión de dependencias indica que **las clases de un sistema deben depender de las abstracciones/interfaces y no de las implementaciones concretas**. Esto significa que las clases no deben depender directamente de clases específicas, sino de interfaces o clases abstractas. Esto lo haremos inyectando dependencias en el constructor de la clase, pero estas dependencias serán interfaces o clases abstractas, no clases finales.

Este principio se centra en la inversión de la dirección de las dependencias en un sistema. En lugar de que los módulos de alto nivel dependan de los detalles de los módulos de bajo nivel, ambas partes deben depender de abstracciones, lo que promueve un diseño más flexible y mantenible.

Principio de Inversión de Dependencias

Algunos conceptos clave:

Los **módulos de alto nivel** suelen ser componentes o clases que encapsulan la lógica principal de una aplicación. Los **módulos de bajo nivel** suelen ser componentes o clases que realizan tareas específicas y están destinados a ser utilizados por los módulos de alto nivel.

- Los módulos de alto nivel no deben depender de los módulos de bajo nivel, **ambos tiene que depender de abstracciones** .
- Las abstracciones no deben depender de los detalles. **Los detalles deben depender de abstracciones** .



Principio de Inversión de Dependencias

Principales ventajas:

- **Flexibilidad** : Al depender de abstracciones en lugar de detalles concretos, el sistema se vuelve más flexible y menos propenso a cambios drásticos cuando se modifican los módulos de bajo nivel.
- **Mantenibilidad** : Cambiar o actualizar los módulos de bajo nivel no debería afectar los módulos de alto nivel si se siguen las abstracciones adecuadas.



Principio de Inversión de Dependencias

Principales ventajas:

- **Reutilización** : Las abstracciones permiten la reutilización de código a través de la implementación de diferentes módulos de bajo nivel que cumplen con la misma interfaz.
- **Pruebas** y **Mocking** : Facilita las pruebas unitarias y el uso de objetos simulados (mocks) para pruebas, ya que se pueden crear implementaciones de abstracciones que se comporten de manera controlada.



¿Alguna consulta?





Resumen

¿Qué logramos en esta clase?



- ✓ Reconocimos la importancia de la cohesión y el acoplamiento en el diseño de clases.
- ✓ Aplicamos principios para mejorar la mantenibilidad y reutilización del código.
- ✓ Estudiamos y ejemplificamos los principios SOLID: SRP, OCP, LSP, ISP y DIP.
- ✓ Refactorizamos ejemplos de clases Java aplicando responsabilidad única y apertura/cierre.



¡Ponte a prueba!

Momento de ejercitación

Te invitamos a aprovechar esta última sección del espacio sincrónico para realizar de manera individual las **actividades disponibles en la plataforma**. Estas propuestas son clave para afianzar lo trabajado y **forman parte obligatoria del recorrido de aprendizaje**.

👉 **Análisis de caso** ————— 👉 **Selección Múltiple**

👉 **Comprensión lectora**

Si al resolverlas surge alguna duda, compartela o tráela al próximo encuentro sincrónico.

< **¡Muchas gracias!** >

